

Priority Queues

What Is a Priority Queue?

A priority queue maintains a dynamic set S of elements, where each element has: - a key (priority), $x.key$ - associated data, $x.data$

In this note, we use a **max-priority queue**, where larger keys mean higher priority.

Core operations: - **MAX()**: return the element with the largest key - **EXTRACT-MAX()**: return and remove the element with the largest key - **INCREASE-KEY(x, k)**: set $x.key = k$, assuming $k \geq x.key$ - **INSERT(x)**: add x to the set, so $S = S \cup \{x\}$

Applications

- Scheduling and event handling
- Shortest paths in [[graphs]] (Dijkstra's algorithm)
- [[Minimum Spanning Tree (MST)]] (Prim's algorithm)
- Compression (Huffman's algorithm)

Linked List Implementations

Unsorted Doubly Linked List

Maintain S in a doubly linked list with no ordering constraint.

![[doubly-linked-list-ex.png]]

Operation behavior: - **MAX()**: linear scan to find the largest key - **EXTRACT-MAX()**: linear scan to find largest key, then remove it - **INCREASE-KEY(x, k)**: update key in place - **INSERT(x)**: insert at the front (assuming $x \notin S$)

Complexity

Let $n = |S|$.

Operation	Time
MAX()	$O(n)$
EXTRACT-MAX()	$O(n)$
INCREASE-KEY(x, k)	$O(1)$
INSERT(x)	$O(1)$

Space: $O(n)$

Sorted Doubly Linked List

Maintain S in a doubly linked list sorted by decreasing key.

![[doubly-linked-list-sorted-ex.png]]

Operation behavior: - $\text{MAX}()$: return first element - $\text{EXTRACT-MAX}()$: remove and return first element - $\text{INCREASE-KEY}(x, k)$: update key, then move x to restore sorted order - $\text{INSERT}(x)$: find correct position, then insert

Complexity

Let $n = |S|$.

Operation	Time
$\text{MAX}()$	$O(1)$
$\text{EXTRACT-MAX}()$	$O(1)$
$\text{INCREASE-KEY}(x, k)$	$O(n)$
$\text{INSERT}(x)$	$O(n)$

Space: $O(n)$

Heap Implementation

A binary max-[[Heaps|heap]] gives a strong overall tradeoff by keeping the maximum at the root and restoring heap order using `BubbleUp` and `BubbleDown`.

Pseudocode

```
MAX()
    return H[1]
```

```
EXTRACT-MAX()
    r = H[1]
    H[1] = H[n]
    n = n - 1
    BubbleDown(1)
    return r
```

```
INSERT(x)
    n = n + 1
    H[n] = x
    BubbleUp(n)
```

```
INCREASE-KEY(i, k)
```

```
H[i] = k
BubbleUp(i)
```

Complexity

Let $n = |S|$.

Operation	Time
MAX()	$O(1)$
EXTRACT-MAX()	$O(\log n)$
INCREASE-KEY(i , k)	$O(\log n)$
INSERT(x)	$O(\log n)$

Space: $O(n)$

Summary

Each implementation optimizes different operations: - Unsorted list: very fast updates/inserts, slow max queries - Sorted list: fast max queries, slow updates/inserts - Binary heap: balanced logarithmic performance for update/insert/extract

In practice, binary heaps are often preferred for general-purpose priority queues.

Related

- [[Amortized Analysis]]
- [[Minimum Spanning Tree (MST)]]